Przetwarzanie języka naturalnego

Sprawozdanie z laboratorium nr 2

Od tekstu do tokenów: n-gramy, sprowadzanie form, odległość edycyjna i tokenizacja podsłowowa

Michael Moses, Nikodem Niemiec

Materiały. Tekst polski: „Pan Tadeusz” (Wolne Lektury, 445 726 znaków). Tekst angielski: Hamlet (Project Gutenberg, ebook 27761 — wydanie z didaskaliami i przypisami edytora, 202 253 znaków). Narzędzia: Python, nltk, spaCy (pl_core_news_sm), stop-words, textdistance, tokenizers, transformers, matplotlib. Wszystko darmowe i uruchamiane lokalnie.

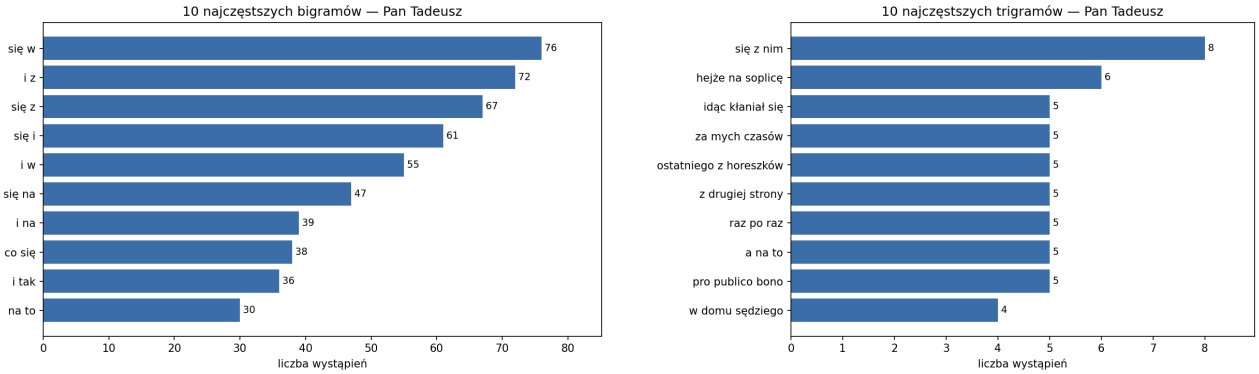
1. Część A — n-gramy w „Panu Tadeuszu”

Tekst oczyszczono (NFC, małe litery, usunięcie cyfr i znaków specjalnych poza . ! ?, zwinięcie białych znaków), podzielono na zdania (nltk, model polski) i na wyrazy (ciągi liter).

wielkość	wartość
zdań	3 984
wyrazów (tokenów)	68 202
wyrazów unikalnych	18 766
średnia długość zdania	17,1 wyrazu

n-gram	wszystkich	unikalnych	udział 10 najczęstszych	występujących raz (% unikalnych)
unigramy	68 202	18 766	16,4%	63,1%
bigramy	68 201	58 804	0,8%	92,7%
trigramy	68 200	67 395	0,1%	99,1%

Histogramy



Rys. 1. Dziesięć najczęstszych bigramów (z lewej) i trigramów (z prawej). Histogram unigramów: wyniki/A_unigramy.png.

Analiza histogramów i odpowiedzi na pytania

Jakie bigramy dominują? Wyłącznie połączenia słów funkcyjnych: „się w” (76), „i z” (72), „się z” (67), „się i” (61), „i w” (55), „się na” (47). Słowo „się” występuje w ośmiu z dwudziestu najczęstszych bigramów. Bigramy z rzeczownikiem pojawiają się dopiero niżej: „na koniec” (29), „w domu” (25). Najczęstszy bigram pojawia się zaledwie 76 razy na 68 tys. — rozkład jest bardzo płaski.

Czy trigramy pokazują zwroty charakterystyczne dla autora? Częściowo. Najczęstszy trigram „się z nim” (8) jest składniowy, ale w czołówce są zwroty bardziej swoiste dla utworu i epoki: „hejże na Soplicę” (6, okrzyk), „za mych czasów” (5), „ostatniego z Horeszków” (5), „pro publico bono” (5), „raz po raz” (5). Kilka trigramów o liczności 4 („z wieku mu”, „wieku mu i”, „mu i z”, „i z urzędu”, „urzędu ten zaszczyt”…) to w rzeczywistości jedna powtarzana fraza („z wieku mu i z urzędu ten zaszczyt należy, idąc kłaniał się…”) rozbita na nakładające się trigramy, a nie osobne zwroty. Wniosek ostrożny: liczności są małe (maksymalnie 8), więc to raczej powtarzane formuły i refreny niż statystycznie mocny „odcisk autora”.

Jak różni się rozkład unigramów od bigramów? Unigramy mają skoncentrowany rozkład: 10 słów (i, w, się, z, na, nie, jak, do, a, że) to 16,4% tekstu, a słowo „i” ma 2128 wystąpień. W bigramach 10 najczęstszych to tylko 0,8% tekstu, a 92,7% bigramów występuje dokładnie raz (dla trigramów — 99,1%). Wraz ze wzrostem n liczba możliwych kombinacji rośnie szybciej niż tekst, więc dane stają się coraz rzadsze (problem rzadkości danych). Histogram unigramów ma wyraźny „ogon” typu Zipfa, histogram trigramów jest niemal płaski.

2. Stemming i lematyzacja — definicje i podsumowanie metod

Stemming to heurystyczne obcinanie końcówek (sufiksów) wyrazu, dające rdzeń (stem). Rdzeń nie musi być poprawnym wyrazem (np. „citi", „welcom"), a algorytm nie zna słownika ani części mowy. Jest szybki i prosty, ale pomija formy nieregularne („was", „went") i bywa niedokładny.

Lematyzacja to sprowadzenie wyrazu do lematu, czyli formy podstawowej ze słownika (np. „was" → „be", „książkami" → „książka"). Wymaga słownika i zwykle znajomości części mowy, dlatego jest wolniejsza, ale daje poprawne wyrazy i radzi sobie z formami nieregularnymi.

metoda	zasada	w laboratorium
tokenizacja zdań/wyrazów	podział na zdania (model Punkt) i na wyrazy	nltk, wyrażenia regularne
n-gramy	ciągi n sąsiednich słów	zip, collections.Counter
stemming Porter	reguły obcinania sufiksów w kilku krokach, ostrożny	nltk.PorterStemmer
stemming Lancaster	iteracyjne reguły, bardzo agresywny	nltk.LancasterStemmer
stemming Snowball	ulepszony Porter, 16 języków (bez polskiego)	nltk.SnowballStemmer
lematyzacja EN	słownik WordNet + część mowy	nltk.WordNetLemmatizer + pos_tag
lematyzacja PL	model statystyczny + reguły (spaCy)	pl_core_news_sm
odległości napisów	liczba operacji edycyjnych / podobieństwo znaków	textdistance
stop words	usunięcie najczęstszych słów funkcyjnych	stop-words (PL)
BPE	iteracyjne scalanie najczęstszych par symboli	tokenizers (Hugging Face)

3. Część B — sprowadzanie form w tekście angielskim

Z Hamleta (tekst z didaskaliami) wylosowano 5 zdań (seed=2), pomijając przypisy i didaskalia. Tokenizacja: nltk.word_tokenize; stemming: Porter; lematyzacja: WordNet z częścią mowy z nltk.pos_tag.

• Will you walk out of the air, my lord?

• You are welcome: but my uncle-father and aunt-mother are deceived.

• Do they hold the same estimation they did when I was in the city?

• Here is, therefore, a play upon the words.

• We are arrant knaves, all; believe none of us.

token	rdzeń (Porter)	lemat (WordNet)	token	rdzeń (Porter)	lemat (WordNet)
are	are	be	welcome	welcom	welcome
deceived	deceiv	deceive	estimation	estim	estimation
did	did	do	was	wa	be
city	citi	city	is	is	be
therefore	therefor	therefore	believe	believ	believe
us	us	u	words	word	word
knaves	knave	knave	walk	walk	walk

Pełna tabela (48 tokenów) jest w pliku wyniki/czesc_B.txt. Stemming i lematyzacja różniły się dla 11 unikalnych słów, m.in.:

- **was**: rdzeń „wa" (algorytm obciął końcówkę „-s"), lemat „be" — lematyzator zna czasownik nieregularny, stemmer nie.
- **city**: rdzeń „citi" (reguła zamiany -y → -i), lemat „city" — rdzeń nie jest wyrazem, lemat tak.
- **welcome / deceived / believe**: rdzenie „welcom", „deceiv", „believ" (obcięte końcowe „-e"), lematy „welcome", „deceive", „believe".
- **are, is, did**: stemmer nic nie robi, lematyzator wskazuje „be" i „do".
- Błąd lematyzatora: **us** → **u** (WordNet traktuje „us" jak liczbę mnogą rzeczownika „u"), co pokazuje, że lematyzacja też zawodzi.

4. Część C — odległość Levenshteina

Dla „Ala ma kota” i „Ala posiada kota” (11 i 16 znaków) **textdistance.levenshtein** daje **6** (znormalizowana odległość 0,375). Własna implementacja programowania dynamicznego daje ten sam wynik.

Wspólny początek „Ala ” i końcówka „ kota” nie zmieniają się. Różni się środek: „ma” (2 znaki) i „posiada” (7 znaków). Operacje: **5 wstawień** (p, o, s, i, a) i **1 zamiana** (m → d), razem 6. Pięć wstawień wynika z różnicy długości (7 − 2). W „posiada” nie ma litery „m”, a jedyny wspólny znak to „a”, więc jeden znak musi zostać zamieniony. Mniej niż 6 operacji się nie da — długości różnią się o 5, a środki mają tylko jedną wspólną literę. Zapisany rozkład operacji to jeden z kilku równie dobrych (zamiana m→d mogła wypaść przy innej literze).

Odległość liczona jest na znakach, a nie na słowach: operacja „ma” → „posiada” nie jest jedną operacją, bo Levenshtein nie zna pojęcia słowa (spacje to też znaki).

5. Część D — porównanie stemmerów (Porter i Lancaster)

Porównanie na całym Hamlecie (33 046 tokenów, 1353 słowa występujące co najmniej 3 razy). Stemmery dały różne rdzenie dla **521 słów (39%)**.

słowo	Porter	Lancaster	słowo	Porter	Lancaster
his	hi	his	thy	thi	thy
your	your	yo	will	will	wil
have	have	hav	very	veri	very
was	wa	was	father	father	fath
are	are	ar	enter	enter	ent
thee	thee	the	heaven	heaven	heav

Który wynik jest sensowniejszy? W większości przypadków **Porter**. Zachowuje rdzenie bliskie wyrazom („have”, „will”, „father”, „heaven”), a Lancaster ucina je do niezrozumiałych skrótów („yo”, „hav”, „fath”, „ent”). Lancaster bywa lepszy tylko tam, gdzie Porter błędnie obcina „-s” z wyrazu, który nim nie jest („his” → „hi”, „was” → „wa”, „thy” → „thi”). Ogólnie Lancaster jest agresywniejszy, więc lepiej łączy formy, ale gorzej zachowuje różnice znaczeń.

Nadmierne obcinanie (różne wyrazy → ten sam rdzeń)

stemmer	rdzeń	wyrazy
Porter	thi	this, thy
Porter	natur	nature, natural
Porter	mad	mad, madness
Porter	hast	hast, haste
Lancaster	the	the, thee, theatre
Lancaster	you	you, youth
Lancaster	her	her, here
Lancaster	not	not, note

Najdobitniejszy przykład: **Lancaster: „the” ← the, thee, theatre** — rodzajnik, zaimek i „teatr” dostają ten sam rdzeń. Porter: „hast” (od have) i „haste” (pośpiech) mają wspólny rdzeń. Lancaster znajduje 84 takie rdzenie, Porter 29.

Niedostateczne obcinanie (formy tego samego wyrazu → różne rdzenie)

stemmer	lemat	formy → rdzenie
Porter	be	are → are, was → wa, were → were, been → been, being → be
Porter	do	did → did, does → doe, done → done
Porter	have	had → had, has → ha, have → have
Lancaster	come	came → cam, come → com

Najdobitniejszy przykład: **Porter: czasownik „be”** rozpada się na pięć różnych rdzeni (are, wa, were, been, be). Są to głównie formy nieregularne — żaden stemmer oparty na obcinaniu końcówek ich nie połączy. Może to zrobić wyłącznie lematyzacja.

6. Część E — polszczyzna wymaga innych narzędzi

Fragment: Inwokacja „Pana Tadeusza” (początek „Litwo! Ojczyzno moja!…”), 68 tokenów bez interpunkcji. Tokenizacja i lematyzacja: spaCy z modelem pl_core_news_sm. Pełna tabela: wyniki/czesc_E.txt.

token	lemat	POS	token	lemat	POS
jesteś	być	AUX	dowie	dowiedzieć	VERB
zdrowia	zdrowie	NOUN	stracił	stracić	VERB
matki	matka	NOUN	ludem	lud	NOUN
cudem	cud	NOUN	wiernym	wierny	ADJ
twą	twój	DET	całej	cały	ADJ
tęsknię	tęsknia (błąd)	VERB	płaczącej	płaczić (błąd)	ADJ
ozdobie	ozdobie (błąd)	NOUN	świecisz	świecisz (błąd)	NOUN
powróciłaś	powrócić być	VERB	Ojczyzno	ojczyzno (POS: ADV)	ADV

spaCy poprawnie sprowadza większość form do lematu (przypadki rzeczowników, osoby czasowników, zaimki). Błędy dotyczą archaicznych i poetyckich form („tęsknię”, „płaczącej”, „ozdobie”), wersu zaczynającego się wielką literą oraz wołacza — to efekt małego modelu uczonego na współczesnym języku.

Stemming polski. Stemmer dla polskiego (Stempel) nie został uruchomiony — pakiet pystempel nie był dostępny w środowisku. spaCy nie zawiera stemmera, więc w sprawozdaniu porównujemy lematyzację z angielskim stemmerem zastosowanym do polskich słów.

Angielski stemmer na polskich słowach

słowo PL	SnowballStemmer("english")	lemat (spaCy)
kotami	kotami	kot
przetłumaczalny	przetłumaczalni	przetłumaczalny
Ojczyzno	ojczyzno	Ojczyzno
zdrowie	zdrowi	zdrowie
pięknością	pięknością	piękność
książkami	książkami	książka
biegałem	biegałem	biegać być
łące	łące	łąka

Angielski stemmer prawie nie zmienia polskich słów: 6 z 8 pozostało bez zmian (poza wielkością liter), a zmiany w 2 pozostałych to przypadek („-y” → „-i”, odcięcie „-e”), a nie analiza morfologiczna. Formy „kotami”, „książkami”, „łące” nie schodzą się z formami podstawowymi, więc stemmer nie spełnia swojej funkcji.

Języki obsługiwane przez SnowballStemmer

arabic, danish, dutch, english, finnish, french, german, hungarian, italian, norwegian, porter, portuguese, romanian, russian, spanish, swedish. **Polskiego brak.** Polszczyzna jest językiem silnie fleksyjnym (7 przypadków, rodzaje, liczby, aspekt, zmiany w temacie typu „pies/psa”), więc kilka reguł obcinających końcówki nie wystarcza. Potrzebny jest słownik fleksyjny lub model, stąd w praktyce stosuje się lematyzację (spaCy, Morfeusz).

7. Porównanie stemmingu i lematyzacji dla obu języków

	angielski	polski
stemming	działa rozsądnie (Porter), bo fleksja jest uboga; błędy przy formach nieregularnych (was → wa, be)	w nltk brak (Snowball nie ma polskiego); angielski stemmer niczego nie sprowadza do rdzenia
lematyzacja	WordNet + część mowy daje poprawne lematy (was → be); błąd: us → u	spaCy radzi sobie z większością form, lecz myli się na formach archaicznych i poetyckich
wniosek	stemming wystarcza do prostych zadań (wyszukiwanie), lematyzacja jest dokładniejsza	potrzebna lematyzacja lub dedykowany stemmer; sam stemming angielski jest bezużyteczny

8. Część F — więcej miar podobieństwa

Zasada działania miar

- **Levenshtein** — minimalna liczba wstawień, usunięć i zamian pojedynczych znaków. Przestawienie dwóch liter kosztuje 2.
- **Damerau–Levenshtein** — jak Levenshtein, ale przestawienie sąsiednich znaków to jedna operacja (typowa literówka).
- **Jaro–Winkler** — podobieństwo 0..1 oparte na znakach wspólnych leżących blisko siebie (w oknie ok. połowy długości napisu) i liczbie przestawień; Winkler dodaje premię za zgodny początek (do 4 znaków).
- **Jaccard (bigramy znakowe)** — liczba bigramów wspólnych dla obu napisów podzielona przez liczbę bigramów występujących w którymkolwiek z nich (część wspólna / suma zbiorów); nie zależy od kolejności bigramów.

Podobieństwo znormalizowane (1 = identyczne)

para	Levenshtein	Damerau-Lev.	Jaro-Winkler	Jaccard (bigramy)	rozrzut
Ala ma kota / Ala posiada kota	0,625	0,625	0,847	0,471	0,376
kot / kto	0,333	0,667	0,556	0,000	0,667
tokenizacja / tokenziacja	0,818	0,909	0,982	0,538	0,443
język / jezyk	0,800	0,800	0,880	0,333	0,547
Ala ma kota / kota ma Ala	0,455	0,455	0,697	0,818	0,364
kot / pies	0,000	0,000	0,000	0,000	0,000

Odległości surowe (Levenshtein / Damerau): Ala ma kota → 6 / 6; kot → kto: 2 / 1; tokenizacja → tokenziacja: 2 / 1; język → jezyk: 1 / 1; Ala ma kota → kota ma Ala: 6 / 6; kot → pies: 4 / 4.

Para, dla której miary różnią się najbardziej: „kot” / „kto”

Rozrzut wynosi 0,667: od 0,000 (Jaccard) do 0,667 (Damerau–Levenshtein). Przyczyny:

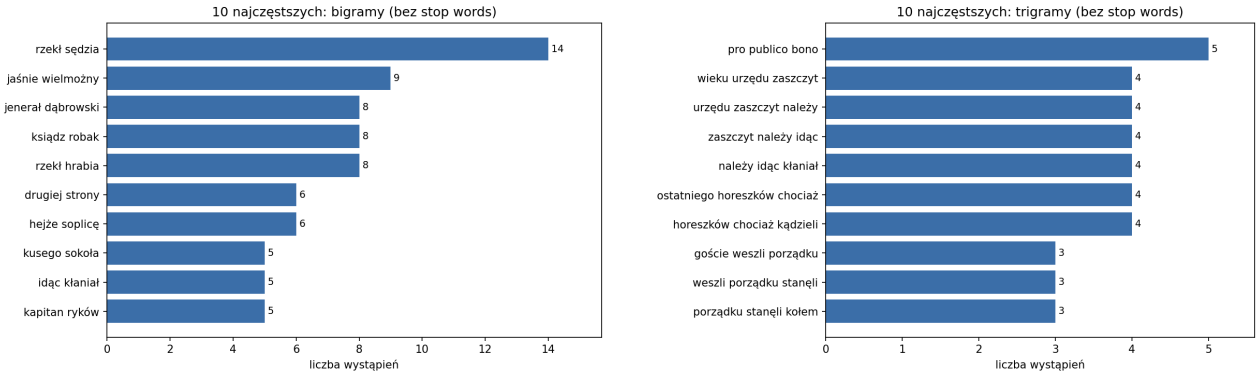
- **Levenshtein (0,333)** traktuje przestawienie jako dwie zamiany (k-**o**-**t** → k-**t**-**o**), czyli koszt 2 z 3 znaków.
- **Damerau–Levenshtein (0,667)** liczy zamianę sąsiednich liter jako jedną operację, więc koszt wynosi 1.
- **Jaro–Winkler (0,556)**: dla trzyznakowych napisów okno dopasowania jest zerowej szerokości, więc liczą się tylko znaki na tych samych pozycjach — pasuje tylko „k”; premia Winklera za wspólny początek nie podniosła wyniku.
- **Jaccard (0,000)**: zbiory bigramów {ko, ot} i {kt, to} nie mają części wspólnej, mimo że napisy składają się z tych samych liter.

Inne obserwacje. Dla „Ala ma kota” / „kota ma Ala” Jaccard daje najwyższą wartość (0,818), a Levenshtein najniższą (0,455) — te same fragmenty znaków w innej kolejności: Jaccard ignoruje porządek, a Levenshtein go karze. Dla „język” / „jezyk” Levenshtein i Damerau widzą jedną zamianę (0,800), Jaro–Winkler pozostaje wysoki (0,880), a Jaccard niski (0,333), bo jedna zmiana litery psuje dwa z czterech bigramów (wspólne zostają tylko „zy” i „yk”). Dla „kot” / „pies” wszystkie miary dają 0.

9. Część G — stop words i nazwy własne

Usunięto stop words (biblioteka stop-words, 335 wyrazów). Z 68 202 wyrazów zostało 43 631 (usunięto 36,0%), ale liczba wyrazów unikalnych spadła tylko z 18 766 do 18 492 (−1,5%).

n-gram	unikalne (z)	unikalne (bez)	udział top10 (z)	udział top10 (bez)	wspólne w top10
unigramy	18 766	18 492	16,44%	2,23%	0
bigramy	58 804	42 727	0,76%	0,17%	0
trigramy	67 395	43 483	0,08%	0,09%	1



Rys. 2. Histogramy bigramów i trigramów po usunięciu stop words. Unigramy: wyniki/G_unigramy_bez_stopwords.png.

Porównanie. Różnice są istotne. Po usunięciu stop words lista 10 najczęstszych unigramów całkowicie się zmienia (0 wspólnych): zamiast „i, w, się, z, na…" są „rzekł" (155), „hrabia" (129), „sędzia" (127), „Tadeusz" (107), „wojski" (89). Bigramy to nazwiska i zwroty fabularne: „rzekł sędzia" (14), „jaśnie wielmożny" (9), „jenerał Dąbrowski" (8), „ksiądz Robak" (8). Udział 10 najczęstszych unigramów spada z 16,4% do 2,2%.

- **Z stop words** lepiej pokazuje **styl**: składnię, spójniki, zaimki zwrotne (np. dominacja „się"), czyli sposób budowania zdań.
- **Bez stop words** lepiej pokazuje **treść**: postacie, tytuły i czynności (rzekł, hrabia, sędzia).
- Uwaga: po usunięciu słów powstają bigramy, które w tekście nie sąsiadowały (np. „hejże soplicę" z „hejże na Soplicę"), a samo usunięcie wymazuje m.in. negację („nie").

Nazwy własne — wyrażenie regularne

(?<=[a-ząćęłńóśźż,;])[A-ZĄĆĘŁŃÓŚŹŻ][a-ząćęłńóśźż]+

Wyrażenie wyszukuje słowo z wielką literą i małymi literami dalej, którego poprzedzają dokładnie: mała litera (albo przecinek lub średnik) i spacja. Dzięki temu wielka litera **w środku zdania** jest odróżniona od wielkiej litery na początku. Zdanie, wers (poprzedza go znak nowej linii) i nagłówki pisane wersalikami („KSIĘGA PIERWSZA") odpadają. Ponadto odrzucono dopasowania krótsze niż 3 litery i słowa z listy stop words (np. „Ty", „Co"). Wynik: 2776 dopasowań, po odsianiu 2761, 922 unikalne nazwy.

miejsce	nazwa	wystąpień
1	Sędzia	89
2	Hrabia	79
3	Wojski	57
4	Tadeusza	50
5	Tadeusz	44
6	Sędziego	38
7	Gerwazy	38
8	Robak	36

miejsce	nazwa	wystąpień
9	Litwie	35
10	Ryków	34

Ocena wyników. Wyrażenie znajduje postacie (Gerwazy, Robak, Ryków, Tadeusz, Telimena, Soplica, Zosia) i miejsca (Litwa). Ograniczenia: (1) **odmiana** — formy jednej nazwy liczone są osobno (Tadeusz 44 + Tadeusza 50; Sędzia 89 + Sędziego 38), więc po połączeniu najczęstszym imieniem byłby Tadeusz; (2) w czołówce są **tytuły i funkcje** (Sędzia, Hrabia, Wojski, Podkomorzy, Klucznik), które w utworze pełnią rolę imion; (3) trafiają się wyrazy niebędące nazwami osobowymi (Bóg, Moskali); (4) nazwy na początku zdania lub wersu są pomijane z założenia.

10. Część H — własny tokenizator podsłowowy (BPE)

Tokenizator BPE (biblioteka tokenizers, ByteLevel) wytrenowano na „Panu Tadeuszu” dla trzech rozmiarów słownika. Przy zamówionych 30 000 tokenów uzyskano tylko **14 154**: korpus jednej książki (ok. 450 kB) ma zbyt mało powtarzających się par (min_frequency=2), więc BPE zakończyło scalanie wcześniej. Pliki: tokenizer_pl_1000.json, tokenizer_pl.json (5000), tokenizer_pl_30000.json.

Podział słów

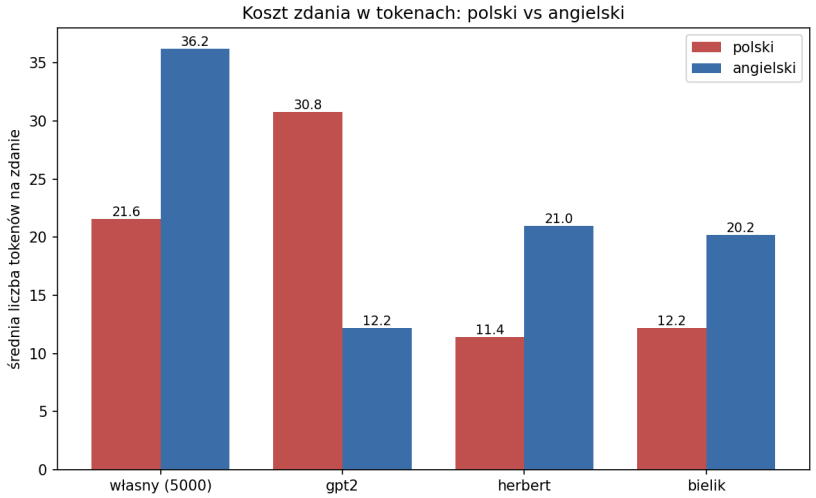
słowo	V = 1 000	V = 5 000	V = 14 154 (zam. 30 000)
nieprzetłumaczalny	9: nie | p | rze | t | łu | ma | cza | l | ny	8: nie | prze | t | łu | ma | cza | l | ny	6: nie | prze | t | łuma | cza | lny
hiperparametryzacja	12: h | i | p | er | pa | ra | me | t | ry | za | c | ja	9: hi | p | er | pa | ra | me | try | za | cja	9: hi | p | er | pa | ra | me | try | za | cja
kotkowi	4: ko | t | ko | wi	2: kot | kowi	2: kot | kowi
Mickiewicz	5: Mi | c | kie | wi | cz	1: Mickiewicz	1: Mickiewicz

Im większy słownik, tym mniej fragmentów. „Mickiewicz” (nazwisko z korpusu) to 5 tokenów przy V=1000 i jeden token przy V=5000. Słowo „hiperparametryzacja” (nieobecne w XIX-wiecznym tekście) pozostaje rozbite na 9 kawałków także przy największym słowniku. „kotkowi” dzieli się na rdzeń i końcówkę („kot | kowi”).

Liczba tokenów na zdanie

język	zd.	wyrazów	własny (5000)	gpt2	herbert	bielik
PL	1	7	25	32	11	11
PL	2	7	13	24	13	12
PL	3	7	20	39	11	12
PL	4	7	30	26	8	10
PL	5	9	20	33	14	16
EN	1	9	37	10	22	23
EN	2	10	25	13	15	15
EN	3	10	42	12	22	20
EN	4	11	41	14	24	23
EN	5	11	36	12	22	20

tokenizator	PL (średnio)	EN (średnio)	PL/EN
własny (5000)	21.6	36.2	0.60
gpt2	30.8	12.2	2.52
herbert	11.4	21.0	0.54
bielik	12.2	20.2	0.60

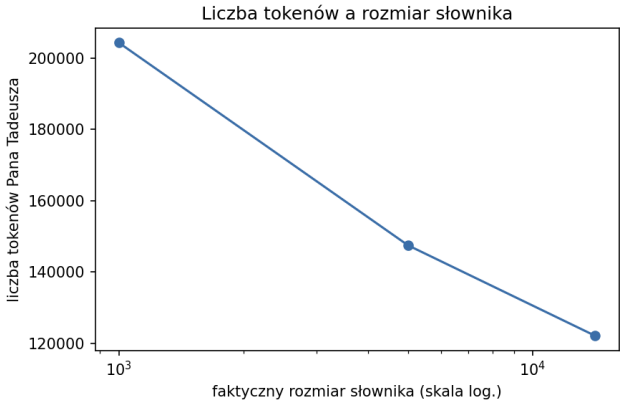


Rys. 3. Średnia liczba tokenów na zdanie dla polskiego i angielskiego (5 zdań każdego języka).

Odpowiedzi na pytania

Który tokenizator jest najbardziej ekonomiczny dla tekstu polskiego, a który dla angielskiego? Spośród zmierzonych: dla polskiego **HerBERT** (11,4 tokenu na zdanie wobec 21,6 dla własnego i 30,8 dla GPT-2), dla angielskiego **GPT-2** (12,2 wobec 21,0 dla HerBERT-a i 36,2 dla własnego). Własny tokenizator jest najgorszy dla angielskiego, bo nie widział angielskich słów i rozbija je na krótkie fragmenty. Bielika nie zmierzono (brak dostępu).

Jak zmienia się liczba tokenów wraz ze wzrostem rozmiaru słownika? Czy zależność jest liniowa? Nie. Cały „Pan Tadeusz” (69 095 wyrazów) to 204 337 tokenów przy V=1 000, 147 477 przy V=5 000 i 122 110 przy V=14 154, czyli 2,96 → 2,13 → 1,77 tokenu na wyraz. Pięciokrotny wzrost słownika (1 000 → 5 000) zmniejsza liczbę tokenów o ok. 28%, a dalszy prawie trzykrotny (5 000 → 14 154) tylko o ok. 17%. To malejące zyski: pierwsze scalenia obejmują bardzo częste pary, kolejne coraz rzadsze (efekt rozkładu Zipfa).



Dlaczego tokenizator wytrenowany na jednej książce wypada dobrze na tej książce, a gorzej na innym tekście? BPE uczy się scalać pary, które często występują w danych uczących. Nasz tokenizator ma w słowniku nazwiska i archaiczne formy z „Pana Tadeusza” (np. „Mickiewicz” to jeden token), a na własnym korpusie potrzebuje 2,13 tokenu na wyraz. Na obcym, angielskim tekście (Hamlet) ta liczba rośnie do 4,05, bo brakuje odpowiednich scaleń i tekst rozpada się na krótkie fragmenty lub pojedyncze bajty. Korpus jest też mały (450 kB), więc słownik jest dopasowany do jednego autora i epoki, a nie do języka w ogóle.

Jaki jest związek z „podatkiem językowym”? Liczba tokenów decyduje o koszcie i długości kontekstu modelu. Tokenizator uczony głównie na angielskim (GPT-2) potrzebuje na polskie zdania średnio 30,8 tokenu, a na angielskie 12,2 — polski jest około 2,5 razy „droższy” (PL/EN = 2,52). Tokenizator uczony na

polskim (HerBERT) odwraca ten stosunek (0,54: angielski jest droższy). To pokazuje, że „podatek” nie wynika z samego języka, tylko z tego, jakich danych użyto do uczenia tokenizatora. Zastrzeżenie: próbka to tylko 5 zdań na język, a zdania PL i EN nie są dokładnymi tłumaczeniami (poza pierwszą parą).

11. Część I — porównanie wszystkich metod

Fragment: pierwsze ok. 20 000 znaków „Pana Tadeusza” (od „Litwo! Ojczyzno moja!”).

wariant	liczba jednostek	rozmiar słownika	jednostek vs wyrazy	słownik vs wyrazy
same wyrazy	3090	1764	100%	100%
lematy (spaCy)	3090	1412	100%	80%
bez stop words	1943	1592	63%	90%
tokeny BPE (V=5000)	5745	1965	186%	111%

Który wariant najmocniej zmniejsza słownik i jakim kosztem?

- **Lematyzacja** zmniejsza słownik najmocniej (1764 → 1412, czyli o ok. 20%) bez zmiany długości tekstu. Koszt: utrata informacji gramatycznej (przypadek, liczba, czas) oraz błędy modelu (np. „tęsknia”, „płaczić”), które tworzą nieistniejące lematy.
- **Usunięcie stop words** skraca tekst o 37% (3090 → 1943 jednostek), ale słownik tylko o ok. 10% (1764 → 1592), bo stop words to mało wyrazów powtarzanych wielokrotnie. Koszt: utrata struktury zdania i negacji („nie”).
- **Tokeny BPE (V=5000)** w tym fragmencie **nie zmniejszają** liczby unikalnych jednostek (1965, czyli 111% słownika wyrazów) i wydłużają tekst o 86% (5745 jednostek). Wynika to m.in. stąd, że BPE zachowuje wielkość liter i interpunkcję (osobne tokeny dla „Ile” i „ile”, „!”, „;”), a warianty „wyrazów” są zapisane małymi literami bez interpunkcji — porównanie nie jest więc całkowicie równe. Zaleta BPE jest inna: słownik jest **stały i skończony** (5000) i nie ma słów nieznanych dla żadnego tekstu, a w naszym fragmencie użyto 1965 z 5000 tokenów tokenizatora (ok. 40%).

Wniosek: żadna metoda nie zmniejsza słownika bez kosztu. Lematyzacja daje największą redukcję bez wydłużania tekstu, kosztem informacji gramatycznej; stop words skracają tekst, lecz słownik prawie zostaje; BPE gwarantuje brak słów spoza słownika i stały rozmiar słownika, kosztem dłuższych ciągów i fragmentów bez własnego znaczenia.

12. Podsumowanie

- Rozkład n-gramów w „Panu Tadeuszu” jest zdominowany przez słowa funkcyjne; wraz z n rośnie rzadkość danych (99,1% trigramów występuje raz).
- Stemming jest szybki, ale daje nieczytelne rdzenie i nie łączy form nieregularnych; lematyzacja jest dokładniejsza, lecz wymaga słownika i modelu.
- Dla polskiego Snowball nie istnieje, a angielski stemmer jest bezużyteczny; potrzebne są narzędzia dedykowane (spaCy).
- Miary odległości różnią się wrażliwością na kolejność znaków; Damerau lepiej traktuje literówki, a Jaccard na bigramach ignoruje kolejność.
- Liczba tokenów zależy od danych, na których uczono tokenizator; zyski ze wzrostu słownika maleją (podatek językowy).